Constitutional Robotics

System Calls as the Programming Model for Actuation, Sensing, and Epistemic World Interaction

The computational core remains effect-pure; world interaction becomes a governed boundary crossing.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Robotics software stacks typically expose physical interaction through driver APIs, middleware message passing, or framework-level abstractions. Actuation and sensing are treated as function calls or topic publications whose authority is implied by runtime position. This conflates control computation with jurisdiction over physical impact and obscures the boundary between internal reasoning and external consequence.

This paper proposes a programming model reform: all reality-touching operations in robotics systems - actuation, sensing, persistence, communication, and time - should be expressed as constitutional system calls. Under this model, interaction with the physical world is mediated by an explicit admissibility function producing structured outcomes: execute, simulate, defer, or refuse. The computational core remains effect-pure; world interaction becomes a governed boundary crossing.

We formalize a minimal robotics syscall interface, define compositional multi-authority chains, introduce legitimacy-state feedback as a first-class signal, and describe a constitutional logging substrate enabling deterministic replay and forensic auditability. Compared to prevalent middleware-based architectures, constitutional robotics centralizes authority evaluation, unifies hardware and simulation pathways, and transforms refusal from an implicit override into a structured, inspectable outcome.

Constitutional robotics reframes robots not as unilateral actors, but as petitioners to influence physical reality under contract.

---

1. The Implicit Authority of Robotics Software

Most robotics systems share a common architectural assumption:

- Controllers compute control signals
- Drivers or middleware deliver those signals to actuators
- Actuators execute

In code, this appears as:

```
motor.set_velocity(0.8)
```

or:

```
publish("/cmd_vel", velocity_msg)
```

Authority is positional. If code executes within the control stack, its commands are presumed legitimate.

Safety mechanisms - limiters, watchdogs, supervisors - are typically layered on top of this assumption. They clamp or override signals but do not reframe the ontology of actuation. The system still treats actuation as a direct extension of computation.

As autonomy increases, this assumption becomes fragile. Controllers may be learned, adaptive, distributed, or economically consequential. The boundary between reasoning and world impact can no longer remain implicit.

---

2. From Driver Call to Constitutional System Call

We propose replacing direct world-touching invocation with a syscall-shaped interface:

```
syscall(ACTUATE_MOTOR, speed=0.8)
```

or

```
syscall(READ_SENSOR, sensor_id=IMU)
```

Under this model:

- The controller produces a proposal
- The syscall boundary evaluates admissibility
- The outcome is structured and typed

Execution is one possible result. It is not the default.

The syscall becomes the canonical interface to physical reality. It is not merely a wrapper around drivers, but a redefinition of the programming primitive for world interaction.

---

3. A Minimal Robotics Syscall Interface

Let:

$$\text{SyscallID} \in \{\text{ACTUATE}, \text{SENSE}, \text{PERSIST}, \text{COMMUNICATE}, \text{TIME}\}$$

Each syscall is a typed request:

```
Syscall {
    id: SyscallID,
    payload: EffectParameters,
    required_authority: Level,
    contract_id: ContractID
}
```

Admissibility is defined as:

$$\text{Admissibility} : \text{Syscall} \times \text{Authority} \to \text{Outcome} \times \text{Reason}$$

Where:

- $\text{Outcome} \in \{\text{execute}, \text{simulate}, \text{defer}, \text{refuse}\}$
- Reason is structured metadata
- Authority encodes capability sets, safety envelopes, and policy constraints

This function is the sole mediator between computational proposals and physical execution.

---

4. Multi-Authority Composition

Robotics systems frequently involve layered authority:

Controller -> Safety Supervisor -> Regulatory Module -> Hardware Interface

In constitutional robotics, this layering is explicit and compositional.

Given authorities $A_1, A_2, \ldots, A_n$:

$$\text{Outcome}_1 = A_1(\text{syscall}) \text{Outcome}_2 = A_2(\text{syscall}) \vdots \text{FinalOutcome} = A_n(\text{syscall})$$

Refusal at any stage terminates the chain with structured reason propagation.

Composition may be sequential, conjunctive, or delegated, depending on authority topology. What matters is that authority evaluation is explicit and auditable.

---

5. Comparison to Middleware Architectures

In ROS-style publish/subscribe systems:

- Nodes broadcast commands
- Subscribers interpret them
- Authority is distributed implicitly across nodes

Safety logic is typically implemented in separate supervisory nodes or embedded within controllers.

Constitutional robotics differs in three key respects:

1. Authority evaluation is centralized at defined syscall boundaries
2. Every world-touching operation is typed and logged
3. Refusal semantics are structured rather than emergent

This shift transforms world interaction from network topology into jurisdictional adjudication.

---

6. Deterministic Replay and Simulation Parity

Simulation is central to robotics development, yet hardware and simulation often diverge due to differences in execution pathways.

Under constitutional robotics:

- Hardware and simulation share the same syscall interface
- Replay engines intercept syscalls and simulate outcomes
- All admissibility decisions are logged

A constitutional log entry may be defined as:

```
ConstitutionalLogEntry = {
    syscall,
    outcome,
    reason,
    authority_id,
    contract_id,
    timestamp
}
```

This log enables:

- Deterministic replay of control sessions
- Audit of refusal and deferral events
- Cross-environment equivalence between hardware and digital twin

Simulation is not a parallel implementation. It is an alternative adjudication of the same boundary.

---

7. Legitimacy-State as First-Class Signal

Constitutional actuation introduces a new observable:

Legitimacy state over time.

Each syscall returns not only execution data but jurisdictional feedback:

- Was the action permitted?
- Under what authority level?
- Why was it refused or deferred?

This feedback becomes a structured signal available to the controller.

Rather than binary success/failure, controllers receive graded epistemic information about admissibility. This enables:

- Adaptive planning under constraint
- Anticipation of refusal patterns
- Authority-sensitive trajectory optimization

The robot gains awareness not only of the physical state of the world, but of the legitimacy state of its own actions.

---

8. Functional Improvements

8.1 Structured Refusal

Traditional safety layers silently clamp or override commands. Controllers may not know why.

Under constitutional robotics:

```
Outcome = refuse
Reason = {zone_limit_exceeded}
```

The controller can respond intelligently:

- Reduce velocity
- Re-plan path
- Enter compliance mode

Refusal becomes actionable information.

8.2 Authority-Aware Adaptation

If:

```
Outcome = defer
Reason = {human_confirmation_required}
```

The system can:

- Switch to safe holding behavior
- Request confirmation
- Adjust proposal to fall within autonomous authority

Adaptation is grounded in explicit jurisdiction feedback.

8.3 Clear Separation of Concerns

Control algorithms operate over physical state and objective functions.

Authority boundaries operate over admissibility contracts.

This separation enables:

- Formal reasoning about control independent of governance
- Static analysis of admissibility constraints
- Clean modularization of safety logic

The controller computes; the boundary adjudicates.

---

9. Real-Time Feasibility

Introducing syscall mediation must not violate real-time guarantees.

Admissibility checks can be:

- Precompiled into constant-time capability checks
- Cached for stable authority states
- Partitioned into fast-path and slow-path logic

Capability systems and verified microkernels demonstrate that boundary checks can be both explicit and performant.

The overhead is architectural, not prohibitive.

---

10. Toward a Robotics Syscall ABI

A concrete robotics syscall ABI would define:

- Canonical identifiers for actuation and sensing
- Structured refusal codes
- Authority level annotations
- Contract versioning semantics

- Logging schemas

Such an ABI could unify:

- Hardware abstraction layers
- Simulation engines
- Replay tools
- Certification artifacts

Control code becomes portable across hardware, simulation, and audit contexts without modification.

---

11. Governance as Structural Backing

This programming model is not aesthetic minimalism.

Physical actuation:

- Alters shared reality
- Carries liability
- May be safety-critical
- Interacts with regulatory regimes

Embedding actuation as a constitutional syscall ensures that:

- No world-touching operation bypasses authority evaluation
- Undefined behavior at the physical boundary becomes structured refusal
- Authority escalation is constrained by explicit contracts

Governance descends into substrate enforcement.

---

12. Appendix A - Epistemic Implications and Learning

Constitutional robotics generates a structured dataset:

```
(Proposal, Outcome, Reason, AuthorityLevel, Timestamp)
```

This dataset enables:

- Learning models that predict admissibility envelopes
- Detection of authority drift over time
- Optimization within refusal likelihood distributions
- Meta-reasoning about policy compliance

Corpus-Wide Reference Positioning

This paper argues that robotics software stacks typically expose physical interaction through driver APIs, middleware message passing, or framework-level

abstractions. Its nearest external comparison fields are cps, governance, security. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Runtime monitors and fallback controllers only protect modeled safety envelopes and can fail through sensor error, timing assumptions, unsafe recovery, or omitted hazards. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[E1] Edward R. Griffor et al. *Framework for Cyber-Physical Systems: Volume 1, Overview.* NIST SP 1500-201, 2017. DOI: 10.6028/NIST.SP.1500-201. https://doi.org/10.6028/NIST.SP.1500-201 **Relationship:** Adjacent cyber-physical systems framework for trustworthiness, timing, data, and lifecycle concerns.

[**E2**] J. Tanner Slagel et al. "A Formal Verification Framework for Runtime Assurance." *NASA Formal Methods*, 2024. NASA NTRS 20230017350. https://ntrs.nasa.gov/citations/20230017350 **Relationship:** Adjacent runtime-assurance formalization, useful for comparing failover safety with constitutional admission and effect recognition.

[**E3**] Nancy G. Leveson and John P. Thomas. *STPA Handbook*. MIT, 2018. https://psas.scripts.mit.edu/home/books-and-handbooks/ **Relationship:** Adjacent systems-theoretic hazard analysis method; used to test treatment of unsafe control and system-level constraints.

[**E4**] Robert N. M. Watson, Simon W. Moore, Peter Sewell, and Peter G. Neumann. *An Introduction to CHERI*. University of Cambridge Technical Report UCAM-CL-TR-941, 2019. DOI: 10.48456/tr-941. cl.cam.ac.uk source **Relationship:** Adjacent capability architecture demonstrating unforgeable hardware authority and compartmentalization.

[**E5**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E6**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E7**] Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

Ableman corpus

[**A1**] Adam Ableman. *A Constitutional Syscall Architecture for Cyber-Physical Control Systems*. Ableman Corpus current edition; work identity `AMCS-LOCAL-WORK-F46D86D479F89BAC`;
source SHA-256
`5E696B753A10BD3C3D36B675A84B95CCF2234F024A6B277BF121E76BFCBCBD18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Effect Sovereignty*. Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Constitutionally Governed Infrastructure.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-EF5747107D56F516`;
source SHA-256
`9C708BA381108EEA252404DF33F7B3B1648EC9CF5F2ACF00BAB88C6A15540926`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Constitutional Syscalls.* Ableman Corpus current edition; work identity `PAPER.PUB.18737229`;
source SHA-256
`5F14356180812262778F8574C292F4F6B560993042E5358B02EAB9CF0F7DE14C`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Ableman Constitutional Governance.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Governance Engineering.* Ableman Corpus current edition; work identity `PAPER.PUB.18749870`;
source SHA-256
`372169F315D192733856B1F4A3F1038BBC73F57CF3ED9ACABAE72B965A69C8D0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.18750683